

# Implementation and Analysis of the Constant Bandwidth Server in the Linux Kernel

Alfredo Ferreira<sup>1</sup>, Rita Barbosa<sup>2</sup>

<sup>1</sup> Instituto Superior de Engenharia do Porto  
1220962@isep.ipp.pt

<sup>2</sup> Instituto Superior de Engenharia do Porto  
1220841@isep.ipp.pt

**Abstract.** This paper presents the design and implementation of the Constant Bandwidth Server (CBS) as a custom scheduling class within the Linux kernel. Developed in the context of real-time operating systems, the work integrates CBS under an Earliest Deadline First (EDF) infrastructure to provide temporal isolation between hard and soft real-time tasks on single-core systems. The implementation introduces dedicated kernel data structures, high-resolution timer mechanisms for budget enforcement and deadline management, and a set of user-space system calls for server lifecycle control. Correctness is evaluated by comparing an observed execution trace against a theoretically derived timeline, demonstrating conformant scheduling behavior with deviations limited to system-level timing imprecision. The results confirm that soft real-time workloads can be safely co-scheduled alongside hard real-time tasks without compromising system schedulability.

**Keywords:** RTOPR, MESCC, CBS, Kernel

## 1. Introduction

The purpose of this report is to document the development and theoretical analysis of a real-time scheduling enhancement for the Linux kernel. This work is conducted within the scope of the M2 Assignment for the Real-Time Operating Systems Programming (RTOPR) course, part of the Master in Critical Computing Systems Engineering (MESCC) at the Instituto Superior de Engenharia do Porto (ISEP).

The assignment requires the implementation of an advanced scheduling algorithm or resource-sharing mechanism to deepen the understanding of kernel-level task management. This specific project focuses on the Constant Bandwidth Server (CBS) algorithm, to provide temporal isolation and efficient resource reclaiming in single-core systems.

## 2. Theoretical Background

Predictability in real-time systems is traditionally achieved by enforcing timing constraints guaranteed off-line via Worst-Case Execution Time (WCET) estimations. However, in modern architectures, non-deterministic behaviors like caching and pipelining make precise WCET estimation difficult. For applications with high execution time variance, such as multimedia, relying on WCET is inefficient and wastes computational resources. To mitigate this, resource reservation techniques are used to achieve temporal isolation, ensuring that a task's overrun does not interfere with the deadlines of other critical tasks.

### 2.1. The CBS Algorithm

In real-time computing, a "server" is a conceptual scheduling mechanism designed to service aperiodic or sporadic workloads while preserving the temporal guarantees of hard real-time activities. The primary goal of the **Constant Bandwidth Server (CBS)** is to allocate a dedicated fraction of the processor bandwidth to a soft real-time task without requiring explicit, prior knowledge of its WCET.

Under an Earliest Deadline First (EDF) infrastructure, each CBS is characterized by a runtime budget  $c_s$ , a maximum budget allocation  $Q_s$ , and a server period  $T_s$ . The ratio  $U_s = Q_s / T_s$  represents the reserved **server bandwidth**. Jobs  $J_{i,j}$  bound to a specific CBS are assigned dynamic deadlines  $d_{i,j}$  equal to the current server deadline  $d_{s,k}$ , allowing them to be scheduled seamlessly alongside standard hard real-time tasks using the native EDF algorithm.

The runtime mechanics of the CBS algorithm are governed by three primary execution rules:

**Rule 1 (Idle arrival with replenishment):** When a job  $J_j$  arrives at time  $r_j$  and the server is idle, the server evaluates whether the remaining budget can be safely spent without exceeding the reserved bandwidth threshold. If  $c_s \geq (d_{s,k} - r_j)U_s$ , a new deadline is generated as  $d_{s,k+1} = r_j + T_s$ , and the budget is fully recharged to  $Q_s$ .

**Rule 2 (Idle arrival with existing budget):** If the condition in Rule 1 is not met, the job is served using the current budget  $c_s$  and the existing server deadline  $d_{s,k}$ . If a job arrives while the server is active (busy), it is simply enqueued into a pending queue according to a non-preemptive discipline (e.g. FIFO).

**Rule 3 (Budget Exhaustion):** As the served job executes,  $c_s$  is decremented by the exact amount of processor time consumed. If  $c_s$  reaches 0 before the job

completes, the budget is immediately replenished to  $Q_s$  and the server deadline is postponed to  $d_{s,k+1} = d_{s,k} + T_s$ .

Because the server budget is immediately replenished when exhausted, there are no finite intervals where a pending task is left completely unserved. Instead, an overrunning task is broken down into computational “chunks”, each assigned progressively later deadlines. This systematically lowers the task’s priority under EDF during overloads, confining the performance penalty strictly to the misbehaving task.

The robust nature of this mechanism is formally established by the **Isolation Property**:

**Theorem 1:** *Given a set of  $n$  periodic hard tasks with a processor utilization  $U_p$  and a CBS with a processor utilization  $U_s$ , the entire set is schedulable by EDF if and only if:*

$$U_p + U_s \leq 1$$

This theorem proves that soft real-time workloads can be integrated into the same system as critical hard tasks without jeopardizing the system’s feasibility, even if the soft requests drastically exceed their expected load parameters.

## 2.2. Task Set

To evaluate and verify the functional correctness of our CBS implementation, we establish a deterministic hybrid real-time workload configuration. This task set serves two distinct evaluation purposes:

- **Correctness Verification:** Serving as a baseline to mathematically validate that the scheduler handles idle arrivals, budget exhaustions, and deadline postponements exactly as dictated by the CBS algorithm rules.
- **System Trace Comparison:** Generating a definitive, theoretical execution timeline that can be directly cross-referenced against our system’s actual execution trace to detect scheduling discrepancies, implementation overheads, or behavioral non-determinism.

The workload consists of three hard real-time tasks ( $t_1, t_2, t_3$ ) mapped directly to the EDF scheduler, and a soft real-time task ( $s_1$ ) bound to a dedicated CBS instance.

**Table 1.** Task set for the CBS algorithm.

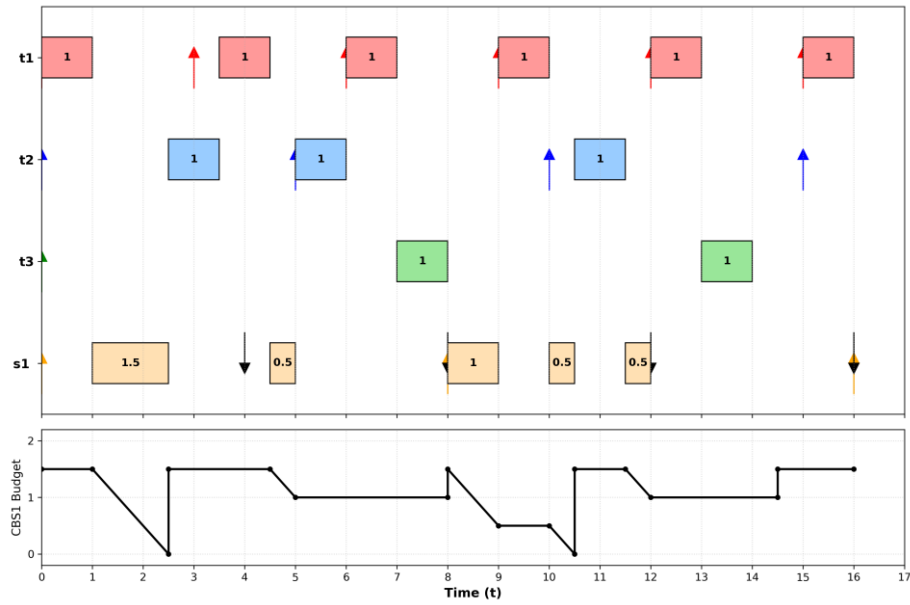
| Task                  | $C_i$ | $T_i = D_i$ | Arrival Instant |
|-----------------------|-------|-------------|-----------------|
| Hard Task 1 ( $t_1$ ) | 1 s   | 3 s         | 0 s             |
| Hard Task 2 ( $t_2$ ) | 1 s   | 5 s         | 0 s             |
| Hard Task 3 ( $t_3$ ) | 2 s   | 20 s        | 0 s             |
| Soft Task 1 ( $s_1$ ) | 2 s   | 8 s         | 0 s             |
| CBS                   | 1.5s  | 4s          | -               |

$$Total\ System\ Allocation = U_p + U_s = \frac{1}{3} + \frac{1}{5} + \frac{2}{20} + \frac{1,5}{4} = 1 \leq 1$$

As specified by Theorem 1, the total combined utilization does not exceed processor capacity, verifying that the task set is theoretically schedulable under an ideal EDF system.

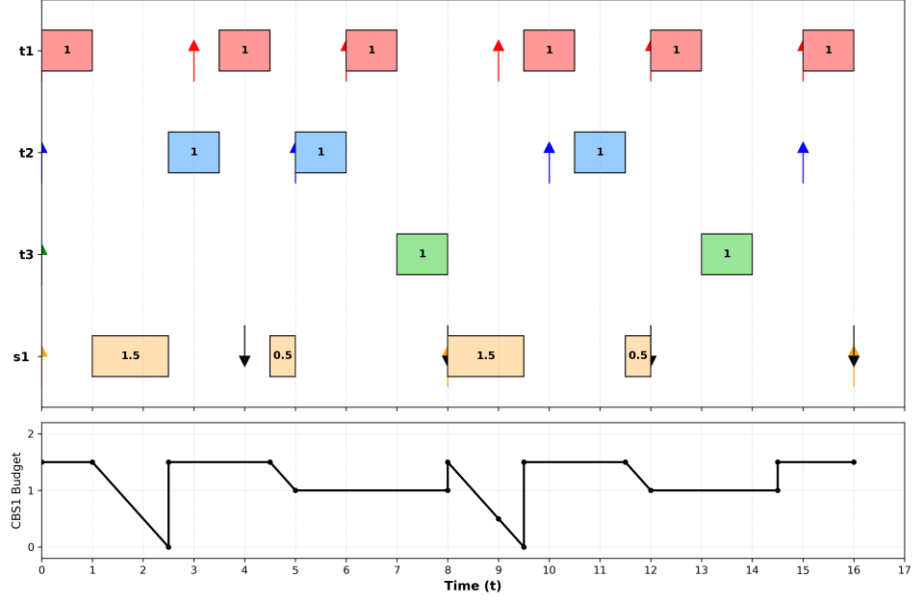
The following timeline illustrates the expected execution trace under an ideal, correct CBS implementation, of the first 15 seconds.

**Diagram 1.** Execution trace of the first 15 seconds.



Given that tasks  $t_1$  and  $s_1$  possess identical deadlines at instant 9s, alternative execution traces are possible. The preemption of task  $s_1$  by  $t_1$  shown in the diagram represents only one potential outcome. Alternatively, task  $s_1$  could continue running without being preempted as shown in Diagram 2.

**Diagram 2.** Alternative execution trace showing no preemption of task  $s_l$  at  $t = 9s$ .



### 3. Implementation Details

This section outlines the architectural assumptions, design constraints, and defensive mechanisms implemented within the kernel space to implement the Constant Bandwidth Server (CBS) scheduling class.

#### 3.1. Implementation Assumptions

When deploying and interacting with the custom scheduling class, a baseline expectation of proper utilizing of the user-space system calls is required. While defensive programming principles were strictly employed during the engineering of these system calls, comprehensive negative test cases and boundary-error injections were not exhaustively executed.

Additionally, the underlying scheduling infrastructure assumes that all hard real-time tasks operate under an implicit deadline model where deadlines are exactly equal to their periods ( $D_i = T_i$ ), mirroring the configuration demonstrated in Table 1.

### 3.2. Data Structures

To facilitate the core mechanics of the CBS algorithm, several custom data structures were developed to store and manage the necessary scheduling parameters. Central to this infrastructure is the global EDF/CBS run queue, `struct edf_cbs_rq`, which manages both standard tasks and active server instances. This run queue contains the following structural components:

- **tasks\_tree**: A balanced Red-Black tree (`struct rb_root`) that maintains all active tasks sorted according to their absolute deadlines under the EDF scheduling discipline.
- **task**: A direct pointer (`struct task_struct *`) referencing the currently executable task with the highest priority (the earliest deadline) in the run queue.
- **servers**: A separate dedicated Red-Black tree (`struct rb_root`) configured to track and manage all registered CBS server instances within the system.
- **server\_count**: An integer value tracking the total number of CBS servers currently allocated in the system.
- **lock**: A raw spinlock (`raw_spinlock_t`) utilized to ensure critical-section serialization and thread safety during concurrent run queue modifications.

```
struct edf_cbs_rq {
    struct rb_root tasks_tree;
    struct task_struct *task;
    struct rb_root servers;
    int server_count;
    raw_spinlock_t lock;
};
```

In addition to the global run queue, a dedicated structure was designed to encapsulate the parameters required to manage an individual CBS instance. The `struct cbs_server` tracks the bandwidth allocation, active jobs, and timing constraints for a specific server using the following structural components:

- **id**: A unique identifier utilized for system bookkeeping and tracking individual server instances.
- **queue\_head**: The anchor for a linked list (`struct list_head`) containing all soft tasks currently assigned to this server. This queue maintains a strict FIFO (First-In, First-Out) ordering for pending tasks, conforming to Rule 2.
- **curr**: A pointer (`struct task_struct *`) referencing the specific soft task that is currently being served and executing within this server instance.
- **node**: An embedding node (`struct rb_node`) that integrates this specific server into the global servers Red-Black tree managed by the core run queue structure (`struct edf_cbs_rq`).

- **relDL / absDL**: Represent the server's relative deadline and the dynamically updated absolute server deadline, respectively.
- **utilization**: An integer value between 1 and 99 that represents the reserved bandwidth fraction used during dynamic deadline calculations.
- **rq**: A pointer referencing the core system run queue structure (`struct rq`) to maintain execution context.
- **maximumCapacity / currCapacity**: Track the server's maximum allowable budget and dynamically depleting current budget, respectively.
- **capacityTimerStart / capacity\_active**: Variable used to track the exact timestamp when a task begins drawing from the budget, accompanied by a flag indicating whether active budget consumption is currently taking place.
- **capacityTimer / deadlineTimer**: Two high-resolution timers (`struct hrtimer`) engineered to handle asynchronous kernel callbacks for budget depletion events (Rule 3) and server deadline expirations. Crucially, the `deadlineTimer` is utilized to prevent stale ordering within the global EDF tree. By managing deadline expirations asynchronously, it enhances system coherence between dynamic CBS deadline updates and scheduler preemption decisions.

```

struct cbs_server {
    u32 id;
    struct list_head queue_head;
    struct task_struct *curr;
    struct rb_node node;
    u64 relDL;
    u64 absDL;
    u32 utilization;
    struct rq *rq;
    u64 maximumCapacity;
    u64 currCapacity;
    u64 capacityTimerStart;
    bool capacity_active;
    struct hrtimer capacityTimer;
    struct hrtimer deadlineTimer;
};

```

### 3.3. Algorithm

The implementation introduces a new scheduling class, `edf_cbs`, which serves as the central mechanism for scheduling both hard real-time EDF tasks and soft real-time tasks managed through CBS. The class is registered with a `queue_mask` value of 16, positioning it above the existing `sched_deadline` scheduling class in the modified scheduler ordering, a design choice made to mitigate task starvation issues encountered during development with alternative policy placements. The scheduling class is declared via `DEFINE_SCHED_CLASS(edf_cbs)` and integrates with the Linux scheduler core through a set of EDF/CBS-specific callbacks, covering task

enqueueing, dequeueing, preemption, task selection, yielding, and several compatibility hooks mandated by the scheduler interface.

The scheduling class is implemented primarily in `kernel/moker/sched.c`, with its core data structure being the EDF red-black tree described in Section 3.2. Tasks are inserted into the tree by the utility function `insert_edf_tree()`, which orders them by absolute deadline. The function `update_edf_pick()` maintains a cached reference to the leftmost node, ensuring that the highest-priority task, the one with the earliest deadline, is always efficiently accessible.

```
void insert_edf_tree(struct rq *rq, struct task_struct *p)
{
    struct rb_root *root = &rq->edf_cbs.tasks_tree;
    struct rb_node **link = &root->rb_node;
    struct rb_node *parent = NULL;
    struct sched_edf_cbs_entity *se = &p->edf_cbs;

    /* Skip if already in tree (already enqueued) */
    if (!RB_EMPTY_NODE(&se->node))
        return;

    /* Initialize the rbtree node for first insertion */
    RB_CLEAR_NODE(&se->node);

    /* Standard BST search to find insertion point */
    while (*link) {
        struct sched_edf_cbs_entity *entry;

        entry = rb_entry(*link, struct sched_edf_cbs_entity, node);
        parent = *link;

        if (se->absDL < entry->absDL)
            link = &(*link)->rb_left;
        else
            link = &(*link)->rb_right;
    }

    /* Link the new node and rebalance the red-black tree */
    rb_link_node(&se->node, parent, link);
    rb_insert_color(&se->node, root);

    /* Update runqueue count */
    add_nr_running(rq, 1);
}

void update_edf_pick(struct rq *rq)
{
    struct rb_node *first = rb_first(&rq->edf_cbs.tasks_tree);

    if (first) {
        struct sched_edf_cbs_entity *first_entity;
        first_entity = rb_entry(first, struct sched_edf_cbs_entity,
node);
        rq->edf_cbs.task = container_of(first_entity, struct
task_struct, edf_cbs);
    } else {
        rq->edf_cbs.task = NULL;
    }
}
```

```
}
```

The enqueue path, implemented in `enqueue_task_edf_cbs()`, distinguishes between hard and soft real-time tasks. Hard EDF tasks are inserted directly into the EDF tree using their own absolute deadline. Soft tasks, by contrast, are managed through their associated CBS server: upon arrival, the server's current deadline is assigned to the task, which is then inserted into the EDF tree accordingly. The CBS idle-arrival rule is also enforced at this stage. If the server's remaining capacity is insufficient to meet its current deadline, the deadline is postponed and the capacity is replenished. This mechanism provides temporal isolation, ensuring that soft tasks cannot consume processor time beyond what their server parameters permit.

```
static void enqueue_task_edf_cbs(struct rq *rq, struct task_struct
*p, int flags)
{
    raw_spin_lock(&rq->edf_cbs.lock);
    if (p->edf_cbs.isHardRT == true)
        enqueue_hard_rt_task(rq, p);
    else
        enqueue_soft_rt_task(rq, p);

    /*If the current running task has a later deadline, request reschedule
    */
    if (rq->curr && rq->curr->policy == SCHED_EDF_CBS) {
        if (rq->curr->edf_cbs.absDL > p->edf_cbs.absDL) {
            resched_curr(rq);
        }
    }

    raw_spin_unlock(&rq->edf_cbs.lock);

#ifdef CONFIG_MOKER_TRACING
    moker_trace(ENQUEUE_RQ, p, -1);
#endif
}

void enqueue_hard_rt_task(struct rq *rq, struct task_struct *p)
{
    insert_edf_tree(rq, p);
    update_edf_pick(rq);
}

void enqueue_soft_rt_task(struct rq *rq, struct task_struct *p)
{
    struct cbs_server *server;
    struct cbs_queue *member;
    bool was_empty;
    u64 arrival_plus_c_over_u;
    u64 c_over_u;

    server = lookup_assigned_cbs_server(&rq->edf_cbs,
                                       p->edf_cbs.cbs_server_id);
    if (!server)
        return;

    p->edf_cbs.absDL = server->absDL;

    /* Already the active task for this server.*/
}
```

```

        if (server->curr == p) {
            activate_soft_task_in_tree_locked(rq, p, server, "active
curr");
            return;
        }
        was_empty = list_empty(&server->queue_head) && (server->curr
== NULL);

        /*
         * CBS idle-arrival rule:
         * If startInstant + (currCapacity / utilization) <= absDL, do nothing.
         * Otherwise, push absDL by relDL and refill capacity immediately.
         */
        if (was_empty) {
            c_over_u = div64_u64(server->currCapacity,
                                server->utilization);
            arrival_plus_c_over_u = p->edf_cbs.startInstant +
c_over_u;
            if (arrival_plus_c_over_u > server->absDL) {
                server->absDL += server->relDL;
                server->currCapacity = server->maximumCapacity;
                /* This function call will update the server's position in the
server RB-tree if needed. It doesn't do much now but if we implement CASH it
will be useful, just thinking ahead. */
                reinsert_cbs_server_tree_locked(&rq->edf_cbs, server);
            }
        }

        /*
         * If this server is idle then promote directly.
         * This makes the task the server owner immediately, instead of putting
it behind the FIFO queue.
         */
        if (was_empty) {
            server->curr = p;
            activate_soft_task_in_tree_locked(rq, p, server,
"promoted");
            return;
        }

        /*
         * Otherwise, enqueue into the server FIFO. The task will be promoted
when it
         * reaches the head of the FIFO naturally.
         */
        member = kmalloc(sizeof(*member), GFP_ATOMIC);
        if (!member) {
            return;
        }
        member->task = p;
        INIT_LIST_HEAD(&member->node);
        list_add_tail(&member->node, &server->queue_head);

        update_edf_pick(rq);
    }
}

```

The dequeue path, implemented in `dequeue_task_edf_cbs()`, mirrors this distinction. The function acquires the EDF/CBS runqueue lock, dispatches to the appropriate hard or soft-task dequeue logic and updates the cached EDF selection. For soft tasks, dequeuing is more involved: the task must be removed from the EDF tree,

consumed CBS runtime must be accounted for, server ownership must be released when appropriate, and the next waiting task must be promoted from the server's FIFO queue. Notably, the implementation avoids advancing the FIFO queue during internal deadline updates, preventing inadvertent release of server ownership during deadline refresh operations.

```
static bool dequeue_task_edf_cbs(struct rq *rq, struct task_struct
*p, int flags)
{
    bool deadline_update = flags & DEQUEUE_UPDATE_DEADLINE;

    raw_spin_lock(&rq->edf_cbs.lock);

    if (!p->edf_cbs.isHardRT)
        dequeue_soft_rt_task_edf_cbs(rq, p, flags, deadline_update);
    else
        dequeue_hard_rt_task_edf_cbs(rq, p);

    update_edf_pick(rq);

    raw_spin_unlock(&rq->edf_cbs.lock);

#ifdef CONFIG_MOKER_TRACING
    moker_trace(DEQUEUE_RQ, p, deadline_update ?
DEQUEUE_UPDATE_DEADLINE : -1);
#endif

    return true;
}

void dequeue_soft_rt_task_edf_cbs(struct rq *rq, struct task_struct
*p, int flags, bool deadline_update)
{
    struct cbs_server *server;
    bool was_curr = false;

    server = lookup_assigned_cbs_server(&rq->edf_cbs,
p->edf_cbs.cbs_server_id);

    if (!server) {
        return;
    }

    /* Debugging to see what members are in the FIFO queue when this
dequeueing is happening */
    dump_soft_queue_locked(server);

    /*
     * Always remove p from the EDF tree if present.
     * This keeps the runqueue accounting aligned with the task's actual
     * runnable state before any CBS ownership transfer happens.
     */
    if (!RB_EMPTY_NODE(&p->edf_cbs.node)) {
        rb_erase(&p->edf_cbs.node, &rq->edf_cbs.tasks_tree);
        sub_nr_running(rq, 1);
        RB_CLEAR_NODE(&p->edf_cbs.node);
    }

    if (deadline_update) {
        return;
    }
}
```

```

    }

    /* Real dequeue/block/exit:
     * release CBS ownership and stop budget timer.
     */
    if (server->curr == p) {
        /* Real dequeue/block/exit of the active soft task:
         * stop budget timer and charge elapsed runtime.
         */
        release_soft_task_locked(server);
        was_curr = true;
    }

    /*
     * Promote next waiting soft task only on real dequeue.
     * Deadline updates do not release ownership, so they must not advance
     * the FIFO queue.
     */
    if ((was_curr || server->curr == NULL) &&
        !list_empty(&server->queue_head)) {
        promote_next_soft_task_locked(rq, server);
    }
}

void dequeue_hard_rt_task_edf_cbs(struct rq *rq, struct task_struct
*p)
{
    if (!RB_EMPTY_NODE(&p->edf_cbs.node)) {
        rb_erase(&p->edf_cbs.node, &rq->edf_cbs.tasks_tree);
        RB_CLEAR_NODE(&p->edf_cbs.node);
        sub_nr_running(rq, 1);
    }
}

```

Task selection is performed by `pick_next_task_edf_cbs()`, both of which refresh the cached EDF pick and return the task with the earliest absolute deadline. When the selected task is a soft CBS task, the scheduler arms the server's capacity timer to activate budget tracking. This ensures that budget is consumed only while the task is actually running, rather than merely while it resides in the EDF tree. Conversely, when the scheduler switches away from a soft task, its capacity timer is cancelled, and any remaining budget is accounted for accordingly.

```

static struct task_struct *pick_next_task_edf_cbs(struct rq *rq,
struct task_struct *prev,
struct rq_flags *rf)
{
    struct task_struct *task;
    struct cbs_server *server = NULL;
    u64 curr_cap = 0;
    bool soft = false;

    /* Hold lock while checking and updating deadlineUpdate flag */
    raw_spin_lock(&rq->edf_cbs.lock);

    if (prev && prev->policy == SCHED_EDF_CBS &&
        prev->edf_cbs.deadlineUpdate)
    {
        prev->edf_cbs.deadlineUpdate = false;
    }
}

```

```

    update_edf_pick(rq);
    task = rq->edf_cbs.task;

    /* If current task is a soft task and a different task from the one
    with highest priority, cancel the current task and account it's capacity
    timer */
    if (prev && prev != task && prev->policy == SCHED_EDF_CBS &&
        !prev->edf_cbs.isHardRT) {
        struct cbs_server *prev_server =
            lookup_assigned_cbs_server(&rq->edf_cbs,
                                      prev->edf_cbs.cbs_server_id);
        if (prev_server) {
            release_soft_task_locked(prev_server);
        }
    }

    if (task && !task->edf_cbs.isHardRT)
        activate_picked_soft_task_locked(rq, task);

    raw_spin_unlock(&rq->edf_cbs.lock);

    return task;
}

```

The remaining callbacks in the scheduling class are largely lightweight compatibility functions. `yield_task_edf_cbs()` requests rescheduling, `yield_to_task_edf_cbs()` returns false, as explicit yielding to a specific task is unsupported, `balance_edf_cbs()` returns zero, reflecting the absence of custom load balancing, and hooks such as `put_prev_task`, `set_next_task`, `switching_from`, `switching_to`, `switched_from`, `switched_to`, `reweight_task`, `prio_changed`, and `update_curr` are present primarily to satisfy the scheduler-class interface requirements.

CBS server logic is implemented separately in `kernel/moker/edf_cbs_rq.c`, keeping server management decoupled from the main scheduling class callbacks and improving the clarity of CBS-specific timing behavior.

Each CBS server maintains two high-resolution timers, as described in Section 3.2. The capacity timer fires when a running soft task exhausts its allocated budget, upon expiry, the server's capacity is replenished, its deadline is advanced by one relative deadline period, and the task is reinserted into the EDF tree with its updated deadline. This constitutes the primary enforcement mechanism of the CBS. A second timer, the deadline timer, periodically replenishes the server and advances its deadline independently. If the server currently owns a task at the time of expiry, the task is removed and reinserted into the EDF tree to reflect the updated scheduling key, thereby preventing stale ordering and maintaining coherence between CBS deadline updates and scheduler preemption decisions.

User-space interaction with the scheduler is exposed through custom MOKER system calls defined in `kernel/moker/syscalls.c`. For hard real-time tasks, `setup_moker_edf_cbs_task(id, startInstant, deadline)` initializes the calling task's EDF/CBS entity, sets its absolute deadline to `startInstant + deadline`, marks it as a hard real-time task, and transitions it to `SCHED_EDF_CBS` via `sched_setscheduler()`.

Soft real-time execution requires a CBS server to be created prior to task attachment. The system call `create_moker_cbs_server(relDL, capacity, start_instant, utilization)` validates the provided utilization value, instantiates a CBS server on the current task's runqueue, and returns the generated server identifier to user space. Utilization values of zero, 100, or greater than 100 are rejected, as utilization is represented as an integer percentage strictly within the open interval (0, 100).

A task is subsequently attached to a server via `setup_moker_edf_cbs_soft_task(server_id, task_id, startInstant, relDL)`, which verifies the existence of the target server, initializes the task's EDF/CBS entity, stores the server identifier, marks the task as soft real-time, assigns the server's current absolute deadline as the task's scheduling deadline, and transitions the task to `SCHED_EDF_CBS`. This establishes the fundamental distinction between hard EDF tasks, scheduled directly by their own deadline, and soft CBS tasks, whose scheduling deadline is derived from the server.

The system call `delay_edf_cbs_task_until_next_T()` provides the mechanism for periodic execution in user space. For hard EDF tasks, it refreshes the task deadline and suspends execution until the next release instant. For soft CBS tasks, it additionally accounts for consumed CBS runtime and releases server ownership where appropriate before refreshing the task period and sleeping until the next activation. This call is thus used by periodic user-space programs to voluntarily suspend execution at the end of each job while preserving the scheduler's EDF/CBS accounting invariants.

Finally, `destroy_moker_cbs_server(server_id)` removes a CBS server from the runqueue. It cancels both high-resolution timers, removes any server-owned task from the EDF tree, invalidates server associations for waiting tasks, frees all FIFO queue entries, removes the server from the server tree, and releases the allocated memory. This completes the user-space lifecycle of a CBS server: creation, task attachment, periodic execution, and explicit destruction upon completion.

## 4. Evaluation

To validate the implemented scheduling policy, a user-space program was developed that exercises the custom system calls described in Section 3.3, instantiating the task set defined in Section 2.2. The resulting execution trace is then compared against the expected trace diagram presented in that section, with the goal of verifying that the EDF/CBS scheduler behaves in accordance with theoretical predictions.

It is important to acknowledge, however, that an exact match between the observed and expected traces is not guaranteed. Two primary sources of deviation must be considered.

First, the Linux kernel executes several background tasks concurrently with any user-space program. Kernel threads, interrupt handlers, and other system activity may transiently preempt or delay the scheduled tasks, introducing timing discrepancies that are outside the control of the implemented policy.

Second, the periodic sleeping mechanism relies on `schedule_hrtimeout()`, which, while offering nanosecond-precision in its API, does not guarantee nanosecond-accurate wakeups. Although this function sets the internal timer slack to zero, the actual wakeup latency remains subject to hardware timer resolution, IRQ latency, scheduler latency, and overall CPU load. When high-resolution timer mode is active, the Linux `hrtimer` subsystem provides finer granularity than the traditional jiffy-based timer wheel, nevertheless, its own documentation acknowledges that resolution and accuracy are inherently system-dependent. Consequently, tasks may wake slightly later than their intended release instants, causing the observed trace to diverge from the ideal by small but non-negligible timing offsets.

These limitations should be kept in mind when interpreting the results presented in the following subsections.

#### 4.1. Tracing Results and Comparison

The following excerpt corresponds to the execution trace captured during the user-space program run, using the task set defined in Section 2.2. Each trace entry follows the format `time, event, number, policy, prio, pid, moker_id, state, name` where the relevant events are: `ENQ_RQ` (task enqueued), `DEQ_RQ` (task dequeued), `SWT_TO` (switched to), `SWT_AY` (switched away), and `SCH_TK` (scheduler tick). Timestamps are expressed in nanoseconds.

|    |                                                 |
|----|-------------------------------------------------|
| 1  | 72661151070,ENQ_RQ,*-1*,16,120,230,1,512,test   |
| 2  | 72661158899,ENQ_RQ,*-1*,16,120,231,2,512,test   |
| 3  | 72661168520,ENQ_RQ,*-1*,16,120,232,3,512,test   |
| 4  | 72661181280,ENQ_RQ,*-1*,16,120,233,100,512,test |
| 5  | 72661191409,SWT_TO,*-1*,16,120,230,1,0,test     |
| 6  | 72661191480,SCH_TK,*-1*,16,120,230,1,0,test     |
| 7  | 73661221864,DEQ_RQ,*-1*,16,120,230,1,1,test     |
| 8  | 73661227814,SWT_AY,*-1*,16,120,230,1,1,test     |
| 9  | 73661227864,SWT_TO,*-1*,16,120,233,100,0,test   |
| 10 | 73661227984,SCH_TK,*-1*,16,120,233,100,0,test   |
| 11 | 75161238306,SWT_AY,*-1*,16,120,233,100,0,test   |
| 12 | 75161238526,SWT_TO,*-1*,16,120,231,2,0,test     |
| 13 | 75161238646,SCH_TK,*-1*,16,120,231,2,0,test     |
| 14 | 75661091703,ENQ_RQ,*-1*,16,120,230,1,512,test   |
| 15 | 76161267781,DEQ_RQ,*-1*,16,120,231,2,1,test     |
| 16 | 76161269770,SWT_AY,*-1*,16,120,231,2,1,test     |
| 17 | 76161269821,SWT_TO,*-1*,16,120,230,1,0,test     |
| 18 | 76161269930,SCH_TK,*-1*,16,120,230,1,0,test     |
| 19 | 77161294375,DEQ_RQ,*-1*,16,120,230,1,1,test     |
| 20 | 77161301255,SWT_AY,*-1*,16,120,230,1,1,test     |
| 21 | 77161301326,SWT_TO,*-1*,16,120,233,100,0,test   |
| 22 | 77161301476,SCH_TK,*-1*,16,120,233,100,0,test   |
| 23 | 77661089862,ENQ_RQ,*-1*,16,120,231,2,512,test   |
| 24 | 77661341493,DEQ_RQ,*-1*,16,120,233,100,1,test   |
| 25 | 77661343173,SWT_AY,*-1*,16,120,233,100,1,test   |
| 26 | 77661343223,SWT_TO,*-1*,16,120,231,2,0,test     |
| 27 | 77661343463,SCH_TK,*-1*,16,120,231,2,0,test     |
| 28 | ...                                             |

Lines 1–4 show tasks 1, 2, and 3 (hard real-time) and task 100 (soft real-time, managed by CBS server S1) being enqueued in rapid succession at program start. At this point, all tasks are inserted into the EDF red-black tree. Hard real-time tasks receive an absolute deadline equal to their arrival instant plus their relative deadline, task 100 inherits the current absolute deadline of its CBS server, consistent with the enqueue behavior described in Section 3.3.

Lines 5–8 confirm that task 1 is selected first, as it holds the earliest deadline among the ready tasks. It executes without preemption and is dequeued upon completion, consistent with the EDF scheduling order expected from the task set.

Lines 9–13 show that upon task 1's completion, the scheduler selects task 100. Task 100 is not dequeued at the point where it is switched away from (line 11), instead, the switch away occurs without a preceding `DEQ_RQ` event, indicating that the CBS server's capacity timer expired, triggering an immediate capacity replenishment and deadline postponement. The task is then reinserted into the EDF tree with its updated deadline, after which task 2 is promoted as the next highest-priority task.

Lines 14–22 cover the second execution window of task 2. At line 14, task 1 is re-enqueued upon its next period release. However, task 1 does not preempt task 2, which is consistent with correct EDF behavior: at the moment of task 1's re-arrival, task 2 holds an earlier absolute deadline and therefore retains the CPU. Task 2 completes its execution and is dequeued (line 15), after which task 1 executes again (lines 17–21) and runs to completion uninterrupted. The scheduler then returns to task 100.

Lines 23–27 show task 2 being re-enqueued (line 23) while task 100 is still running. This ordering represents a minor deviation from the expected trace diagram in Section 2.2, attributable to the timing imprecision discussed in Section 4. Nevertheless, the scheduling decision is correct since task 2 does not preempt task 100, as the soft real-time task retains the CPU until it completes its current job. Once task 100 is dequeued (line 24), the scheduler correctly promotes task 2.

Overall, the observed trace is consistent with the theoretical behavior of the EDF/CBS policy. The execution order, preemption decisions, and CBS budget enforcement all conform to expectations, with deviations limited to minor timing offsets arising from system-level interference, as anticipated in Section 4.

## 5. Conclusion

This work presented the design, implementation, and evaluation of a Constant Bandwidth Server scheduling class for the Linux kernel, developed as a custom scheduling policy within the MOKER framework. The implementation integrates seamlessly with the existing Linux scheduler infrastructure by registering a dedicated scheduling class, `edf_cbs`, that manages both hard real-time EDF tasks and soft real-time tasks through the CBS abstraction.

The theoretical foundations of the CBS algorithm, temporal isolation, dynamic deadline assignment, and budget-based enforcement, were translated into concrete kernel mechanisms, including a unified EDF red-black tree, per-server FIFO queues,

and dual high-resolution timer callbacks for capacity enforcement and deadline management. The isolation property guaranteed by Theorem 1 was preserved throughout the implementation, ensuring that soft real-time tasks cannot compromise the schedulability of co-existing hard real-time workloads.

Evaluation against the reference task set defined in Section 2.2 demonstrated that the implemented scheduler behaves in accordance with theoretical predictions. The observed execution trace confirmed correct task ordering under EDF, proper CBS budget exhaustion and replenishment, deadline postponement upon overrun, and appropriate task promotion within server queues. Deviations from the ideal trace were limited to minor timing offsets attributable to kernel background activity and the inherent precision limits of the `schedule_hrtimeout()` mechanism, both of which were anticipated and discussed prior to evaluation.

Future work could extend this implementation in several directions. Support for multi-core systems would require per-CPU run queues and load balancing mechanisms beyond the single-core scope addressed here. The CASH (Capacity Sharing) algorithm, identified as a natural extension of CBS, would further improve resource utilization by enabling idle bandwidth reclamation across servers. Additionally, more exhaustive stress testing and boundary-condition injection would strengthen confidence in the robustness of the user-space system call interface.

Overall, this project demonstrated that resource reservation techniques such as CBS can be practically realized within the Linux kernel, providing a principled foundation for integrating soft real-time workloads into systems with hard timing requirements.

## References

1. L. Abeni and G. Buttazzo, "Integrating multimedia applications in hard real-time systems," in *Proceedings of the 19th IEEE Real-Time Systems Symposium*, 1998, p. 4.
2. Bootlin: Elixir Cross Referencer — Linux kernel source code (v4.5): `schedule_hrtimeout_range_clock`.  
[https://elixir.bootlin.com/linux/v4.5/ident/schedule\\_hrtimeout\\_range\\_clock](https://elixir.bootlin.com/linux/v4.5/ident/schedule_hrtimeout_range_clock). Accessed: May 2026
3. Bootlin: Elixir Cross Referencer — Embedded Linux kernel source browser. <https://bootlin.com>. Accessed: May 2026

## Appendix

- Implementation of system call to initialize the EDF/CBS entity

```
int do_setup_moker_edf_cbs_task(u32 id, u64 startInstant, u64 deadline)
{
#ifdef CONFIG_MOKER_EDF_CBS_POLICY
    struct sched_edf_cbs_entity *sched_entity = &current->edf_cbs;

    RB_CLEAR_NODE(&sched_entity->node);
```

```

    sched_entity->absDL = startInstant + deadline;
    sched_entity->startInstant = startInstant;
    sched_entity->relDL = deadline;
    sched_entity->deadlineUpdate = false; // deadline already defined
    sched_entity->id = id;
    sched_entity->isHardRT = true;
    struct sched_param param = { 0 };
    return sched_setscheduler(current, SCHED_EDF_CBS, &param);
#endif
    return 1;
}

```

- **Implementation of system call to create CBS servers**

```

long do_create_moker_cbs_server(u64 relDL, u64 capacity, u64
start_instant, u32 utilization)
{
#ifdef CONFIG_MOKER_EDF_CBS_POLICY
    struct rq *rq;
    struct rq_flags rf;
    struct cbs_server *server;
    int id;

    if (utilization == 0 || utilization >= 100)
        return -EINVAL;

    rq = task_rq_lock(current, &rf);

    server = create_cbs_server(&rq->edf_cbs, start_instant, relDL,
capacity, utilization);
    if (!server) {
        task_rq_unlock(rq, current, &rf);
        return -ENOMEM;
    }
    id = server->id;

    task_rq_unlock(rq, current, &rf);

    return id;
#endif
    return -ENOSYS;
}

```

- **Implementation of system call to associate a soft task with an existent server**

```

int do_setup_moker_edf_cbs_soft_task(u32 server_id, u32 task_id, u64
startInstant, u64 relDL)
{
#ifdef CONFIG_MOKER_EDF_CBS_POLICY
    struct sched_edf_cbs_entity *sched_entity = &current->edf_cbs;
    struct rq *rq;
    struct rq_flags rf;
    struct cbs_server *server;
    u64 server_absDL;
    struct sched_param param = { 0 };

    rq = task_rq_lock(current, &rf);

    server = lookup_assigned_cbs_server(&rq->edf_cbs, server_id);
    if (!server) {
        task_rq_unlock(rq, current, &rf);
        return -EINVAL;
    }

```

```

    }

    server_absDL = server->absDL;

    task_rq_unlock(rq, current, &rf);

    RB_CLEAR_NODE(&sched_entity->node);
    sched_entity->startInstant = startInstant;
    sched_entity->absT = startInstant;
    sched_entity->relDL = relDL; /* task period/timekeeping */
    sched_entity->absDL = server_absDL; /* CBS scheduling deadline */
    sched_entity->deadlineUpdate = false;
    sched_entity->cbs_server_id = server_id;
    sched_entity->id = task_id;
    sched_entity->isHardRT = false;

    return sched_setscheduler(current, SCHED_EDF_CBS, &param);
#endif
    return -ENOSYS;
}

```

- Implementation of system call that provides the mechanism for periodic execution in user space

```

int do_delay_edf_cbs_task_until_next_T(void)
{
#ifdef CONFIG_MOKER_EDF_CBS_POLICY
    if (edf_cbs_policy(current->policy)){
        ktime_t expires;
        ktime_t now;

        if (current->edf_cbs.isHardRT) {
            ktime_t now = ktime_get();
            ktime_t expires = ns_to_ktime(current->edf_cbs.absDL);

            /* Hard RT:
             * Sleep until the next period deadline, without accounting
             * or releasing CBS server ownership. This is a hard delay,
             * as the task keeps its current scheduling priority and
             * remains at the head of its server's queue if it has one.
             */
            refresh_task_deadline(current);

            if (ktime_compare(expires, now) <= 0){
                current->edf_cbs.deadlineUpdate = true;
                set_tsk_need_resched(current);
                return 1;
            }

            set_current_state(TASK_INTERRUPTIBLE);
            schedule_hrtimeout(&expires, HRTIMER_MODE_ABS);
            __set_current_state(TASK_RUNNING);

            return 0;
        } else {
            struct rq *rq;
            struct rq_flags rf;
            struct cbs_server *server;
            u64 now_before_account;

            /*
             * Soft RT:

```

```

/* Account consumed CBS capacity and release CBS ownership before
 * voluntarily sleeping until the next period.
 * FIFO promotion remains handled by scheduler enqueue/dequeue paths.
 */
now_before_account = ktime_get_ns();
rq = task_rq_lock(current, &rf);

server = lookup_assigned_cbs_server(
    &rq->edf_cbs, current->edf_cbs.cbs_server_id);

if (server && server->curr == current){
    /* Subtracts consumed CBS runtime from capacity */
    account_cbs_runtime(server);

    if (!RB_EMPTY_NODE(&current->edf_cbs.node)){
        rb_erase(&current->edf_cbs.node,
            &rq->edf_cbs.tasks_tree);
        RB_CLEAR_NODE(&current->edf_cbs.node);
        sub_nr_running(rq, 1);
    }

    server->curr = NULL;

    if (!list_empty(&server->queue_head)) {
        struct cbs_queue *next_entry;
        struct task_struct *next;

        next_entry = list_first_entry(
            &server->queue_head,
            struct cbs_queue, node);
        next = next_entry->task;

        list_del(&next_entry->node);
        kfree(next_entry);

        server->curr = next;
        next->edf_cbs.absDL = server->absDL;
        next->edf_cbs.deadlineUpdate = false;

        insert_edf_tree(rq, next);
    }

    update_edf_pick(rq);
}

current->edf_cbs.deadlineUpdate = true;

task_rq_unlock(rq, current, &rf);

/*
 * Refresh this soft task's own period deadline.
 * It keeps its own period bookkeeping, while CBS
 * scheduling priority remains server-deadline based.
 */
refresh_task_period(current);
expires = ns_to_ktime(current->edf_cbs.absT);
now = ktime_get();
if (ktime_compare(expires, now) <= 0) {
    set_tsk_need_resched(current);
    return 1;
}

```

```

        set_current_state(TASK_INTERRUPTIBLE);
        schedule_hrtimeout(&expires, HRTIMER_MODE_ABS);
        __set_current_state(TASK_RUNNING);

        {
            u64 now_ns = ktime_get_ns();
            s64 deltaT = (s64)(current->edf_cbs.absT - now_ns);
            s64 deltaDL = (s64)(current->edf_cbs.absDL - now_ns);
        }
        return 0;
    }
}
#endif
return 0;
}

```

- **Implementation of system call to remove a CBS server from the runqueue**

```

int do_destroy_moker_cbs_server(u32 server_id)
{
    struct cbs_server *server;
    struct cbs_queue *entry, *tmp;
    struct rq *rq;

    rq = cpu_rq(task_cpu(current));

    raw_spin_lock(&rq->edf_cbs.lock);

    server = lookup_assigned_cbs_server(&rq->edf_cbs, server_id);
    if (!server) {
        raw_spin_unlock(&rq->edf_cbs.lock);
        return -ENOENT;
    }

    /*
     * Stop timers before freeing the server.
     *
     * hrtimer_cancel() waits for an active callback to finish, so the
     * server won't be freed while its timer function is still running.
     */
    hrtimer_cancel(&server->capacityTimer);
    hrtimer_cancel(&server->deadlineTimer);

    /* If the current server task is still in the EDF tree, remove it. */
    if (server->curr) {
        struct task_struct *p = server->curr;

        if (!RB_EMPTY_NODE(&p->edf_cbs.node)) {
            rb_erase(&p->edf_cbs.node, &rq->edf_cbs.tasks_tree);
            RB_CLEAR_NODE(&p->edf_cbs.node);
            sub_nr_running(rq, 1);
        }

        p->edf_cbs.cbs_server_id = -1;
        server->curr = NULL;
    }

    /* Free waiting soft tasks from the CBS FIFO. */
    list_for_each_entry_safe(entry, tmp, &server->queue_head, node) {
        if (entry->task)
            entry->task->edf_cbs.cbs_server_id = -1;
    }
}

```

```
        list_del(&entry->node);
        kfree(entry);
    }

    rb_erase(&server->node, &rq->edf_cbs.servers);

    raw_spin_unlock(&rq->edf_cbs.lock);

    kfree(server);

    return 0;
}
```